PROJECT UPDATE · 2026.10

hs / simulate
함수형 사고를 실행 가능한 경험으로

그래프로 만든 게임과 학습 패널까지 왔습니다.
3주 안에 완성하고, 사용자 테스트로 검증합니다.

함수를 연결하고,
결과를 보며 함수형 프로그래밍을 익힙니다.

WHO

기성 프로그래머

기존 개발 경험을 바탕으로 함수형 사고와 추상화 이해

HOW

시각적 실습

노드 연결과 실행으로 함수의 입력·출력·합성을 확인

NEXT

게임 제작

학습한 개념으로 작게라도 동작하는 결과물 완성

하스켈은 왜 어려운가

01 · 추상화

개념이 먼저 온다

Functor·Monoid·Monad가 수학 용어로 먼저 나옵니다. 어디에 쓰는지 느끼기 전에 정의부터 외우게 됩니다.

02 · 타입

오류가 멀리서 터진다

타입 추론 덕분에 타입을 덜 적지만, 오류가 난 위치와 원인이 떨어져 있고 메시지도 어렵습니다.

03 · 문법

읽는 방법이 다르다

공백으로 호출, 커링, `.` · `$`, point-free, 연산자 섹션. 익숙한 언어와 읽는 순서부터 다릅니다.

04 · 순수성

상태를 바꿀 수 없다

변수에 대입하지 않고 새 값을 만듭니다. 게임처럼 상태가 계속 바뀌는 프로그램을 어떻게 짜야 할지 막막합니다.

05 · 지연 평가

언제 계산되는지 안 보인다

필요할 때만 계산하므로 실행 순서가 코드 순서와 다릅니다. 머릿속에 실행 과정을 그리기 어렵습니다.

06 · 결과물

화면까지 멀다

입문 예제는 작은 함수뿐입니다. IO와 라이브러리를 익혀야 비로소 화면에 무언가 나옵니다.

> 결국 **개념을 아는 것과 직접 만드는 것 사이의 간격**이 큽니다.

어려움마다 보이는 방법으로 바꿉니다

어려움	hs / simulate의 방식	현재 화면에서
추상화	추상화한 함수를 다시 재료로 쌓기 · 법칙을 실행해서 확인	클래스를 노드로 끌어오기, 법칙 검사 ✓/✗
타입	연결하는 순간 타입이 보이고, 맞지 않으면 막힘	꺽쇠 사슬 `Model ▸ Model`, 연결 차단
문법	그래프와 하스켈 정의를 토큰 단위로 연결	DEFINITION 패널, 문법 패널 8개
순수성	상태는 `model → model` 함수로만 바뀜	`program`의 model / view / handle / step
지연 평가	필요한 노드만 실행되는 지연 평가 실행기	무한 리스트도 필요한 만큼만 계산
결과물	처음부터 게임 템플릿으로 시작	클릭 카운터·주사위, HTML 하나로 내보내기

현재 진척도: 템플릿과 학습 패널 기준

템플릿 · 게임 예제

커리큘럼 게임 3개 중 — 1 / 3 완성

- **클릭 카운터** — 버튼·타이머·Space 키, 업그레이드 구매, 방치 보상 — 완료
- **주사위** — 순수 난수(StdGen), 그림 위젯 — 완료 · 보조
- **빈 게임** — 새 게임을 시작하는 가장 작은 program — 완료 · 보조
- **숫자 맞히기** — 비교·힌트·시도 횟수 — 2주차
- **슈팅 게임** — 키 입력·이동·충돌 판정 — 3주차

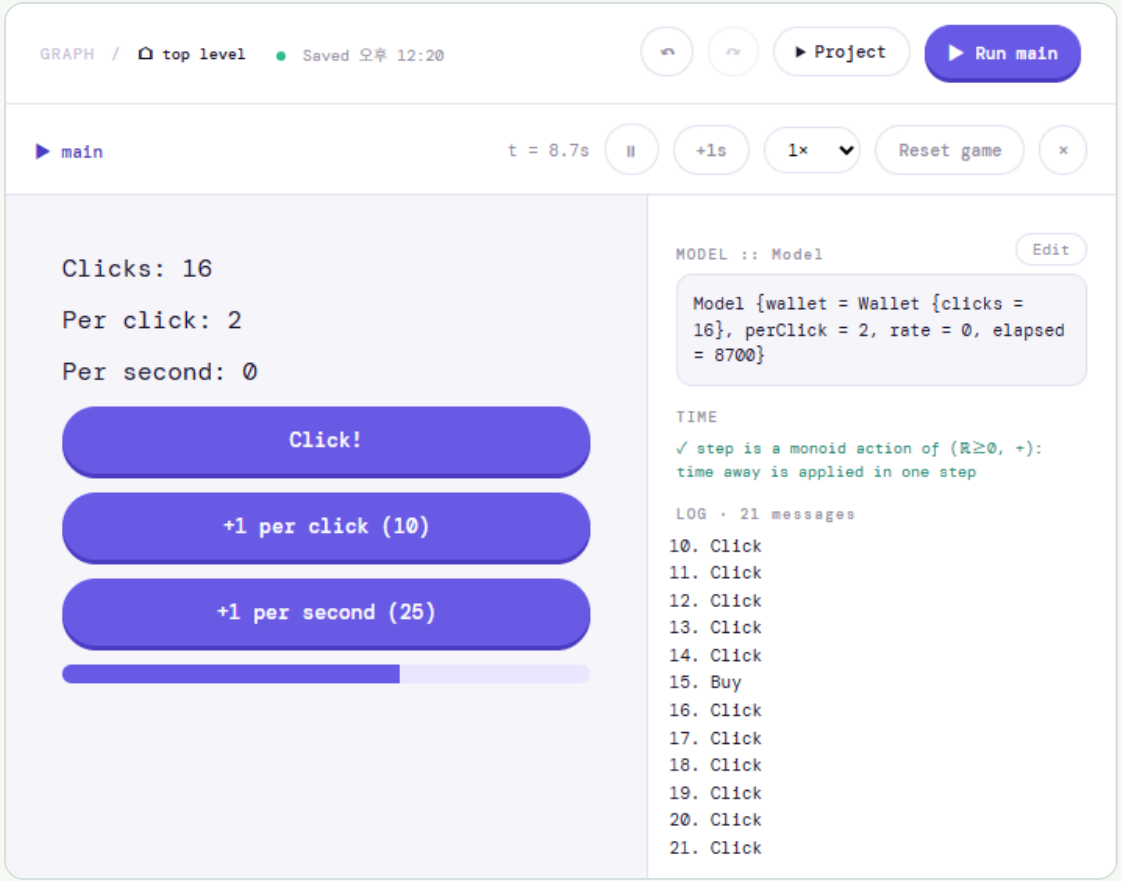
학습 패널 · 문법과 예제

예제 문제 5개 중 채점 연결 — 0 / 5

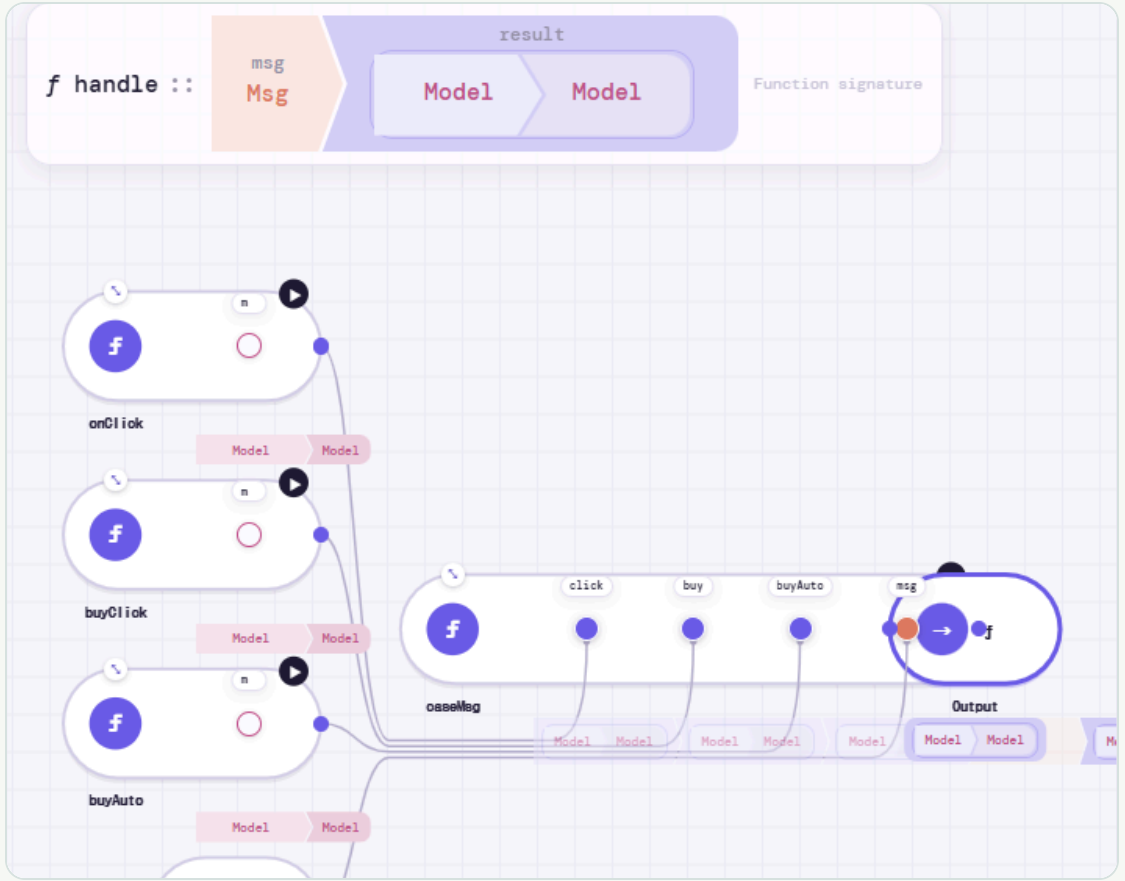
- **문법 8개** — 한국어 설명 · 하스켈 코드 · 해설 — 완료
- **예제 문제 5개 목록** — 두 수 더하기 ~ 함수 합성하기 — 완료
- **예제를 그래프로 풀기** — add·isZero·ifThenElse·compose로 풀 수 있음, 문제와 연결은 안 됨 — 부분
- **문제 열기 · 자동 채점** — 1주차
- **단원 4–8 문법·예제** — 6주차

기반: 단위 테스트 251개 통과 · UI만으로 게임을 만드는 브라우저 e2e 테스트

템플릿: 그래프만으로 만든 게임



클릭 카운터 실행 · 버튼이 메시지를 `handle`로 보내고, 오른쪽에 현재 Model과 메시지 기록이 보입니다.



handle의 본체 · `handle msg = caseMsg onClick buyClick buyAuto msg`를 노드로 만든 그래프입니다.

학습 패널: 문법과 예제 문제



문법 8개

함수 정의와 호출 · 타입과 화살표 · Int와 Integer · Bool과 조건식 · 커링과 부분 적용 · 함수 합성 · 리스트 기초 · 패턴 매칭

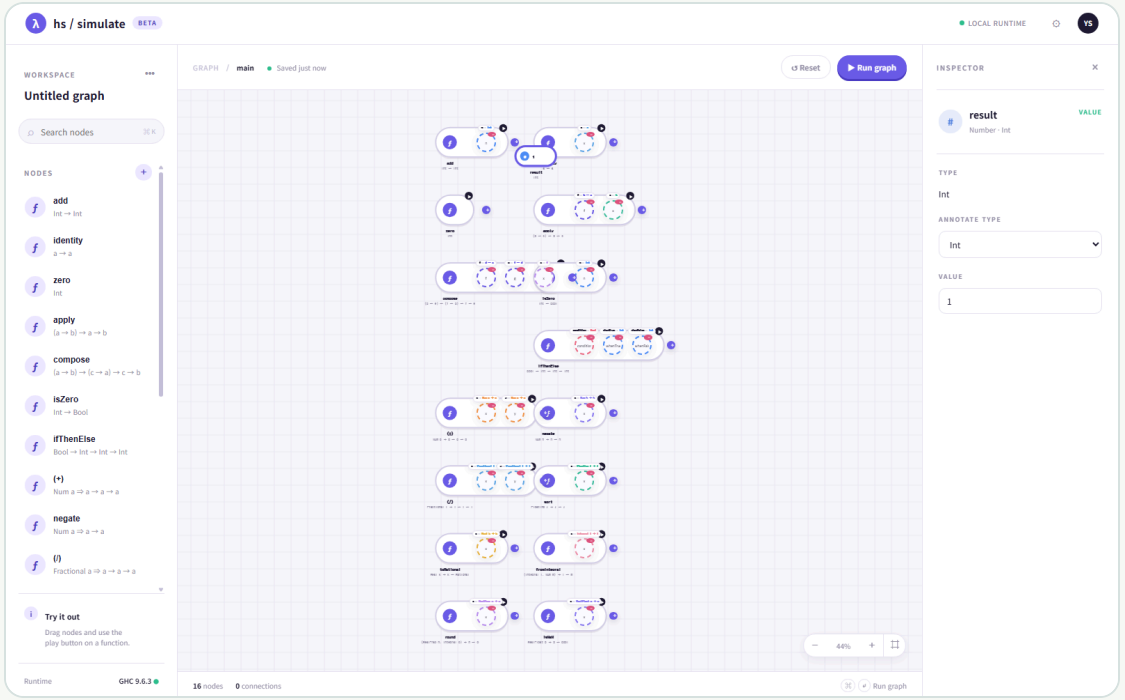
예제 문제 5개

두 수 더하기 · 0인지 판별하기 · 조건에 따라 값 선택하기 · 부분 적용 함수 만들기 · 함수 합성하기

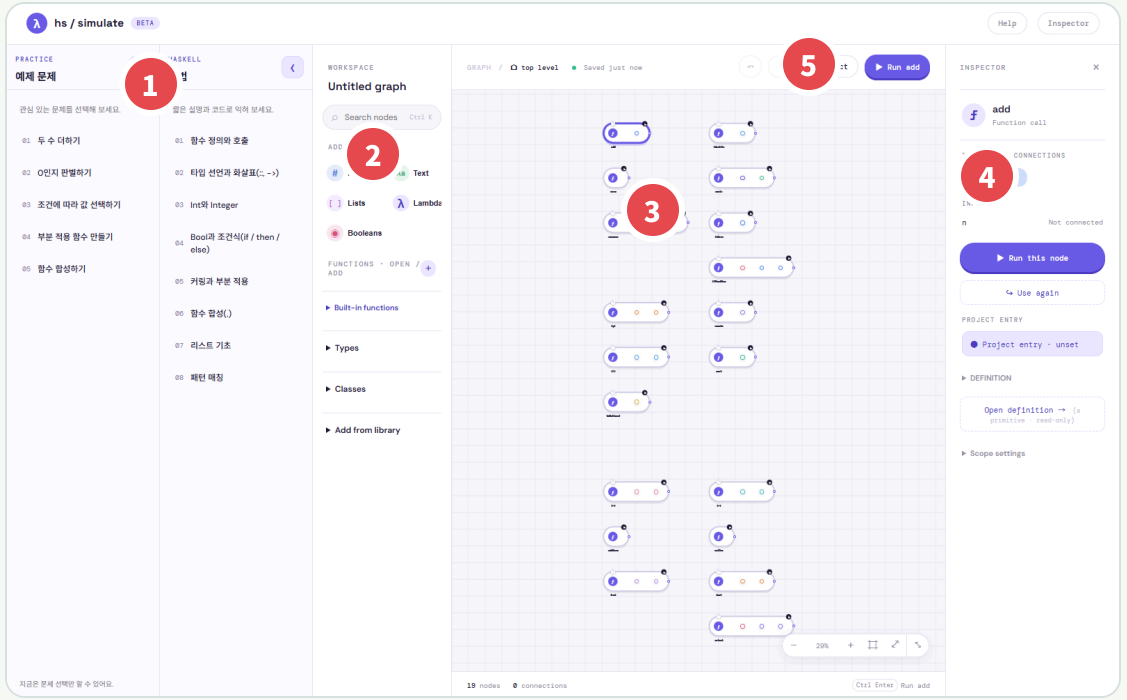
남은 일

예제는 아직 선택만 됩니다. 문제를 열면 그래프 과제가 되고 자동으로 채점되도록 연결합니다(1주차).

지난 발표 화면과 비교하면



이전 · 함수 목록, 캔버스, 값 인스펙터



현재 · 학습 패널부터 인스펙터까지 다섯 영역

1. 예제 문제·문법 학습 패널 추가
2. 작업 공간: 값 추가 · 함수 · Types · Classes · 라이브러리
3. 슬롯은 점, 오른쪽 출력 포트에서 연결선
4. 타입을 꺽쇠 사슬 `Int ▸ Int` 로 표시
5. Project 메뉴: 템플릿 · 저장 · 게임 내보내기

가장 큰 차별점: 추상화를 계속 쌓아가기

함수 자체를 추상화하고, 그 결과를 다시 조합해 더 높은 수준의 함수를 만듭니다.

스크래치 · 엔트리

블록·함수로 동작 재사용

사용자 정의 블록이나 함수로 반복되는 동작을 묶어 다시 사용합니다.

두 도구에도 추상화와 재사용 기능이 있습니다.

hs / simulate의 핵심

추상화한 함수를 다시 추상화

함수를 값으로 전달·합성해 만든 새 함수도, 다음 추상화의 재료로 사용합니다.

조합 방식을 일반화하고, 그 위에 더 큰 규칙을 쌓아갑니다.

작은 함수 → 조합한 함수 → 재사용 가능한 규칙 → 게임 로직

핵심: 각 단계의 결과가 다음 단계의 재료가 되어 추상화가 누적됩니다.

기능 참고: Scratch 사용자 정의 블록 · 엔트리 함수 블록 / 차별화 설명은 본 프로젝트의 설계 방향

역할 분담: 구현과 검증을 병행

메인 개발

양병선

- 예제 채점·튜토리얼·초보자 모드 구현
- 숫자 맞히기·슈팅 게임 템플릿 제작
- 함수·타입 시스템 확장 및 코드 통합
- 사용자 테스트 결과 반영 수정

테스트 및 검증 · 서브 개발

김종면

- 예제 채점 테스트 케이스 작성
- 사용자 테스트 설계·진행·기록
- 템플릿 회귀 확인과 오류 재현
- 문법·예제 콘텐츠와 UI 보완

공동 진행: 매주 하스켈 학습 · 구현 리뷰 · 테스트 결과로 다음 작업 결정

09 · DEVELOPMENT MILESTONES

6주 마일스톤: 3주 완성 → 사용자 테스트 1회

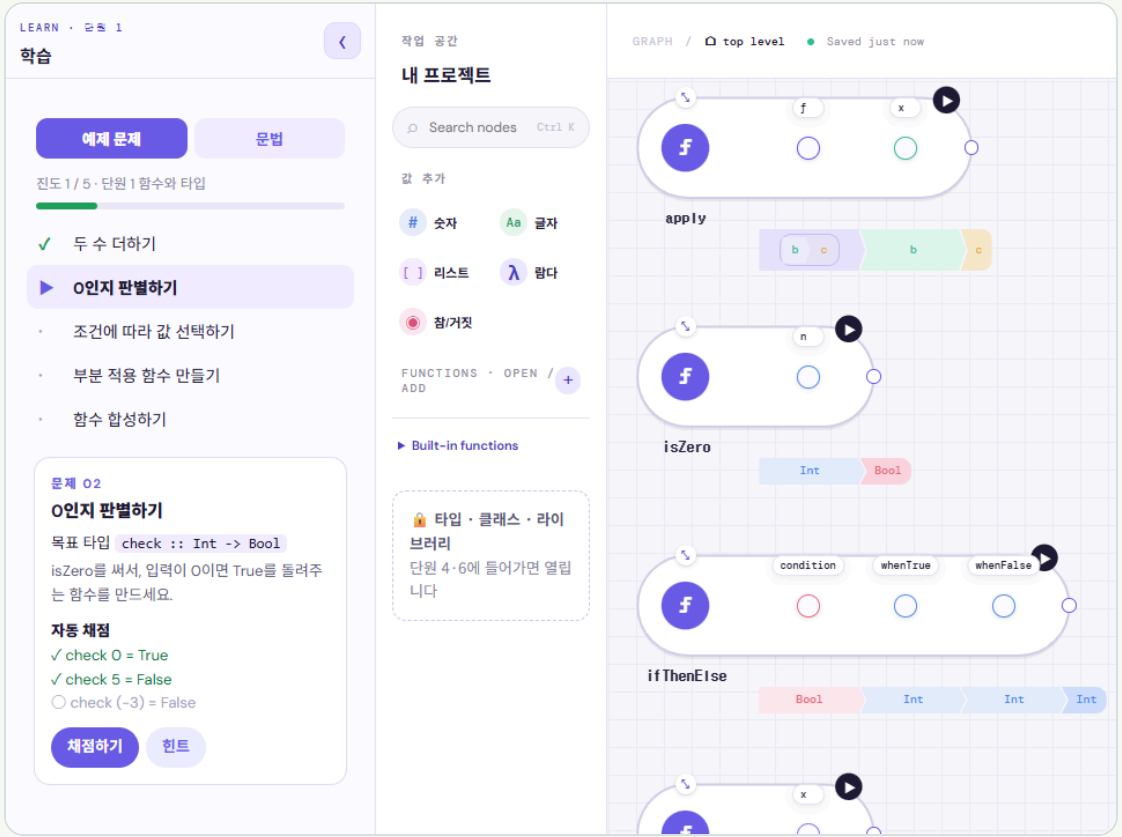
1주 · 개발 | 2주 · 개발 | 3주 · 완성 | 4주 · 사용자 테스트 | 5주 · 반영 | 6주 · 마무리

기간	개발 초점	완료 기준
1주	예제 문제 5개를 그래프 과제로 연결: 문제 열기 → 목표 타입·테스트 입력 → 자동 채점	5문제 모두 풀이·채점 가능
2주	숫자 맞히기 템플릿 · 클릭 카운터 따라 만들기 튜토리얼 · 타입 오류 한국어 안내	튜토리얼만 보고 카운터 완성
3주	슈팅 게임 템플릿 · 초보자 모드와 학습 탭 통합 · 테스트 과제지·설문 준비	**완성: 기능 동결**, 템플릿 3개 동작
4주	사용자 테스트 1회: 기성 프로그래머 5–8명, 1인 60분	과제 기록·설문 수집
5주	테스트에서 막힌 지점 상위 항목부터 수정	심각도 상위 문제 해결
6주	단원 4–8 문법·예제 보강 · 웹 배포 · 최종 시연	수정한 과제 재확인 · 시연

개발과 병행해 매주 하스켈 학습

주차	하스켈 학습 주제	프로젝트 연결
1주	순수 함수 · 타입 서명 · Bool	예제 채점 테스트 케이스 작성
2주	고차 함수 · 커링 · 함수 합성	튜토리얼 단계 설계
3주	대수적 데이터 타입 · 레코드 · Maybe	숫자 맞히기·슈팅의 Model / Msg 설계
4주	리스트 · 재귀 · foldr	적 목록과 충돌 판정 이해 · 테스트 관찰
5주	타입클래스 · 디폴팅	타입 오류 안내 문구 개선
6주	Monoid · Functor · 순수/효과 분리	단원 7–8 콘텐츠 작성

처음 쓰는 사람이 길을 잃지 않게

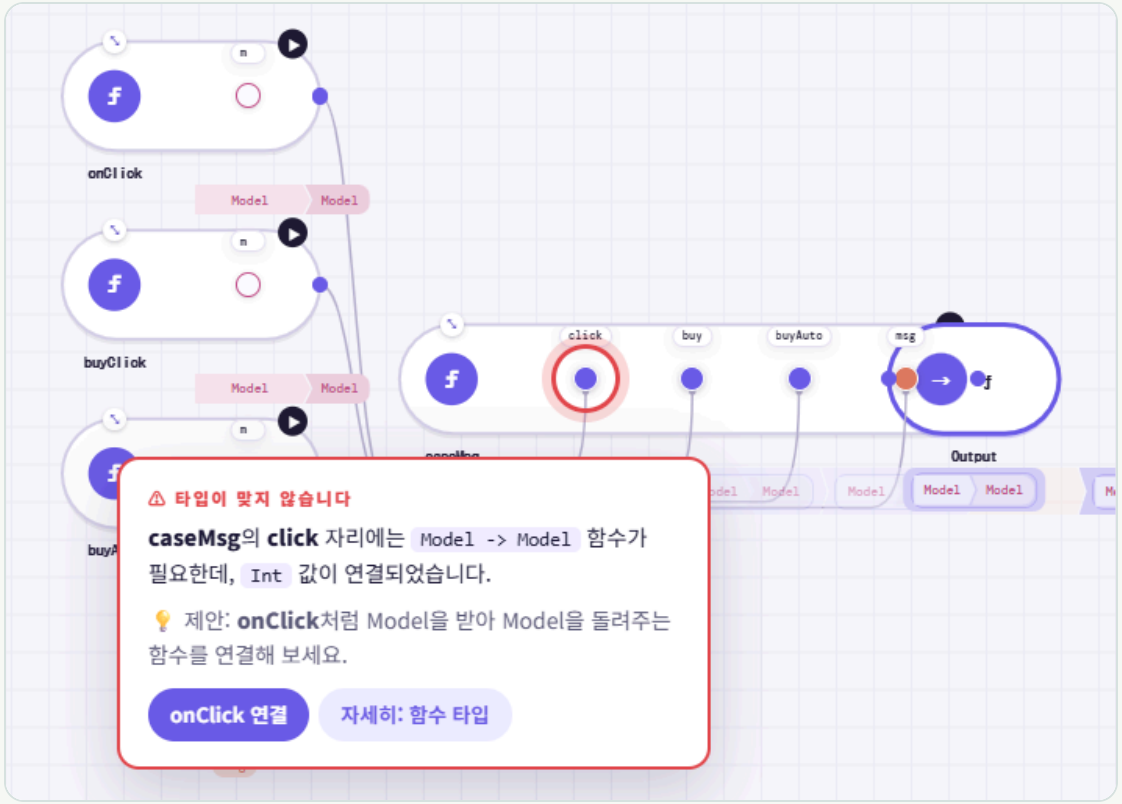


초보자 모드 · 학습 탭 통합 · 예제·문법을 한 탭으로 합치고 진도와 자동 채점을 보여 줍니다. 타입·클래스·라이브러리는 단원에 맞춰 열고, 라벨은 한국어로 통일합니다.

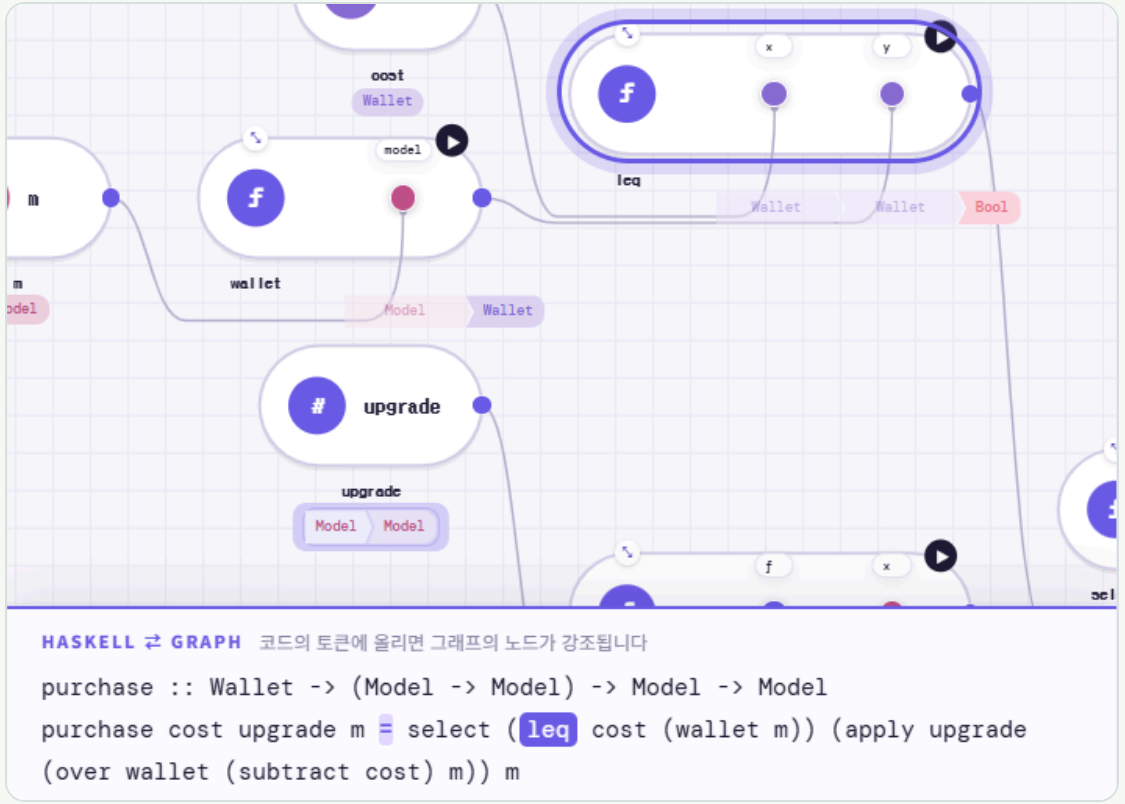


따라 만들기 튜토리얼 · 클릭 카운터를 6단계로 안내합니다. 단계마다 할 일 하나만 강조하고, 같은 내용의 하스켈 코드를 함께 보여 줍니다.

타입과 코드를 그 자리에서 읽히게



타입 오류를 그 자리에서 한국어로 · 막힌 슬롯을 표시하고, 필요한 타입과 들어온 타입, 고칠 방법을 바로 제안합니다.



그래프 ⇄ 하스켈 분할 보기 · 인스펙터 안의 DEFINITION을 캔버스 아래로 넓힙니다. 토큰에 올리면 해당 노드가 강조됩니다.

함께 할 일: 연결선을 타입별 색으로 · 빈 캔버스에 다음 할 일 안내 · 좁은 화면에서는 패널을 탭으로 전환

제작 후 테스트: 자동 검증과 사용자 테스트

매 변경마다 · 자동

기능이 깨지지 않았는지

- 단위 테스트 251개: 타입 엔진 · 실행기 · 법칙 · 템플릿
- e2e: 템플릿 없이 UI만으로 게임을 만들고 플레이
- 템플릿 회귀: 3개 게임의 실행 결과 비교
- 예제 채점기 자체 테스트: 정답·오답 그래프 각각

4주차 · 1회

사람이 실제로 쓸 수 있는지

- **대상** 하스켈 경험이 없는 기성 프로그래머 5–8명
- **형식** 1인 60분, 소리 내어 생각하기, 화면 녹화
- **과제** ① 예제 문제 1–5 풀기
- **과제** ② 튜토리얼로 클릭 카운터 만들기
- **과제** ③ 일부러 넣은 타입 오류 고치기
- **과제** ④ 그래프를 보고 하스켈 정의 읽기

무엇을 재고, 어떻게 반영하는가

지표	측정 방법	목표
과제 완료율	과제 ①–④를 도움 없이 끝낸 비율	80% 이상
소요 시간	튜토리얼로 클릭 카운터를 완성한 시간	20분 이내
막힌 지점	도움 요청·멈춘 위치를 녹화에서 기록	상위 5개 분류
사용성	SUS 설문 10문항	68점 이상
학습 효과	사전·사후 퀴즈: 타입 읽기 · 커링 · 합성	사후 점수 향상

기록
녹화·설문·퀴즈

→

분류
심각도 · 빈도

→

5주차 수정
상위 항목부터

→

6주차 재확인
같은 과제로 확인

교육용 커리큘럼: 모든 단원이 같은 순서

대상은 기성 프로그래머. 8개 단원을 따라가며 하스켈 개념을 그래프로 만들고, 마지막에 게임으로 묶습니다.

① 문법 카드
학습 패널 · 설명과 코드

→

② 그래프 실습
같은 식을 노드로

→

③ 체크포인트
예제 문제 · 자동 채점

→

④ 게임 적용
템플릿의 함수 고치기

→

⑤ 코드로 확인
DEFINITION으로 읽기

PART 1 · 단원 1–3

함수와 타입

지금 있는 문법 8개·예제 5개로 진행

PART 2 · 단원 4–6

데이터와 클래스

타입 편집기·클래스 노드로 실습

PART 3 · 단원 7–8

구조와 게임

Monoid·Functor로 게임 3개 완성

단원 1–4: 함수에서 데이터까지

단원	하스켈 개념	학습 패널	그래프 실습	게임 적용
1 · 함수와 타입	함수 정의, `::`, `->`, Int / Integer	문법 01–03 · 예제 01 두 수 더하기	`add` · `zero` 연결, 시그니처 고정	점수 계산 함수
2 · 조건	Bool, `if then else`	문법 04 · 예제 02 0인지 판별, 03 조건 선택	`isZero`, `ifThenElse`	목표 점수 도달 판정
3 · 커링과 합성	부분 적용, `.`, 람다	문법 05–06 · 예제 04 부분 적용, 05 합성	빈 슬롯 = 부분 적용, `compose`, λ 노드	클릭 규칙 조합 `onClick`
4 · 데이터 타입	`data`, 레코드, Maybe / Either	새 문법: data · 레코드 · Maybe	타입 편집기, 생성자 · 필드 · 분기 함수	게임의 Model / Msg 설계

단원 1–3은 현재 학습 패널의 문법 01–06 · 예제 01–05와 그대로 이어집니다.

단원 5–8: 구조를 쌓아 게임으로

단원	하스켈 개념	학습 패널	그래프 실습	게임 적용
5 · 리스트와 재귀	리스트, 패턴 매칭, `foldr` · `map`	문법 07–08 · 새 예제: 합계, 길이	`[ , , ]`, 정의 그래프 열어보기	화면 `column [...]`, 적 목록
6 · 타입클래스	Eq · Ord · Show · Num, 디폴팅	새 문법: class · instance · deriving	클래스 노드로 제약, deriving	점수 출력 `show`, 비교
7 · 대수 구조	Semigroup · Monoid · Functor, 법칙	새 문법: `<>` · `fmap`	법칙 노드 ✓/✗, 반례 보기	입력 합치기 `every <> onKey`, 그림 겹치기
8 · 게임 프로그램	순수/효과 분리, `program`	새 문법: model · view · update	템플릿 따라 만들기, 시간 여행	카운터 → 숫자 맞히기 → 슈팅, HTML 내보내기

단원 4–8의 새 문법·예제는 6주차에 학습 패널에 추가합니다.

평가: 단원마다 채점, 끝에는 게임

단원 체크포인트

자동 채점 예제

목표 타입과 테스트 입력으로 그래프를 판정합니다. 틀리면 반례와 관련 문법 카드를 보여 줍니다.

읽기 · 쓰기

그래프 ⇄ 하스켈 변환

하스켈 정의를 보고 그래프를 만들고, 그래프를 보고 정의를 씁니다. 그래프 밖에서도 읽을 수 있는지 확인합니다.

최종 결과물

내 게임 하나

템플릿 3개 중 하나를 고쳐 규칙을 바꾸고 HTML로 내보냅니다. 합성·타입·상태 갱신을 그 게임으로 설명합니다.

MVP 완료 기준 — 예제 5개 채점 · 템플릿 3개 플레이 · 사용자 테스트 과제 완료율 80% · 저장한 게임을 다시 열어 같은 동작

3주 안에 완성하고,
사람이 쓰는 모습으로 확인합니다.